

AUTOMATIC NUMBER PLATE RECOGNITION (ANPR) & VEHICLE LOG SYSTEM

A Classical Computer Vision & Persistent Audit Logging Framework

Project Title:	Automatic Number Plate Recognition (ANPR) & Vehicle Log System
Author / Candidate:	Muskan
Course:	Computer Vision (CV) — Build Your Own Project
Platform / Institution:	VITyarthi — BYOP Submission
Academic Year:	2026
Repository URL:	https://github.com/muskan/anpr-system

Abstract: This project presents an end-to-end Automatic Number Plate Recognition (ANPR) and Vehicle Log System built using Python and classical computer vision. The system localizes license plates using edge-preserving bilateral filtering, morphological black-hat transforms, vertical Sobel edge gradients, rectangular morphological closing, and contour aspect-ratio filtering. Detected regions are perspective-corrected using 4-point homography warping and recognized through a multi-engine OCR pipeline with positional letter/digit disambiguation. All detection events, confidence scores, and blacklist flags are stored in an indexed SQLite database and logged to rotating files.

2. Introduction

Automatic Number Plate Recognition (ANPR) is an indispensable technological pillar in modern Intelligent Transportation Systems (ITS). Rapid global vehicle growth has made manual vehicle tracking infeasible for automated toll plazas, controlled access barriers, residential complexes, and highway surveillance networks. ANPR enables automated, non-intrusive logging of vehicle movements by capturing imagery, localizing license plates, extracting alphanumeric registration characters, and checking records against active databases.

While modern deep learning architectures (e.g. YOLO, Faster-RCNN) achieve high accuracy, they impose significant computational burdens, require GPU accelerators, demand large annotated datasets, and lack explainability. This project demonstrates that a carefully engineered pipeline utilizing **classical computer vision** (OpenCV) can localize license plates with sub-150 ms latency on CPU hardware while delivering high geometric accuracy. Paired with an extensible OCR engine, positional disambiguation algorithms, and a relational SQLite audit layer, this system delivers an accessible, production-ready solution.

3. Problem Statement

Conventional manual vehicle registration methods face severe operational constraints:

- **Human Fatigue and Error:** Gate operators frequently misread or mistype alphanumeric plates, particularly characters with visual similarity (e.g., 'O' vs '0', 'l' vs '1', 'B' vs '8').
- **Throughput Bottlenecks:** Manual entry creates severe queuing at toll booths and security checkpoints during peak traffic periods.
- **Absence of Automated Audit Trails:** Paper or spreadsheet logs lack tamper-proof timestamps, cryptographic indexing, or instant blacklist cross-checking.
- **Environmental & Perspective Variability:** Vehicle cameras capture plates at oblique angles, under varying lighting, and with camera vibration.

The objective of this project is to build an autonomous, lightweight Python-based ANPR system capable of ingesting still images, localizing the vehicle plate via classical CV, deskewing the plate using 4-point homography, extracting the registration number, validating plate syntax, flagging blacklisted vehicles, and maintaining a structured SQLite database.

4. Functional Requirements

The system is architected into four fully decoupled functional modules:

Module	Input Structure	Core Operations	Output / Stored Artefacts
Module 1: Plate Localization	Source image path (str) or BGR ndarray	Bilateral filter, Black-Hat transform, Sobel vertical gradient, closing, contour aspect-ratio filter (1.8–7.5), 4-point perspective warp.	PlateDetectionResult (crop ndarray, bbox tuple, polygon pts, confidence, is_detected).

Module 2: OCR Extraction	Perspective-deskewed cropped plate ndarray	Height standardization (120px), CLAHE, multi-variant binarization (Otsu & adaptive), OCR extraction, positional letter/digit disambiguation, regex syntax validation.	OCRResult (clean_text, raw_text, composite confidence, is_valid, status).
Module 3: Vehicle Log & DB	Plate number, confidence, image path, flagged status	SQLite ACID transactions, indexing, date-filtered queries, substring search, watchlist blacklist flagging/unflagging, summary stats, CSV export.	Database records in vehicle_logs table, logs/app.log rotating file.
Module 4: CLI Dashboard	User terminal arguments via argparse	Subcommands: scan, batch-scan, list, search, flag, stats, export. Formats output tables with Rich.	Terminal display cards, benchmark timing, exported CSV files.

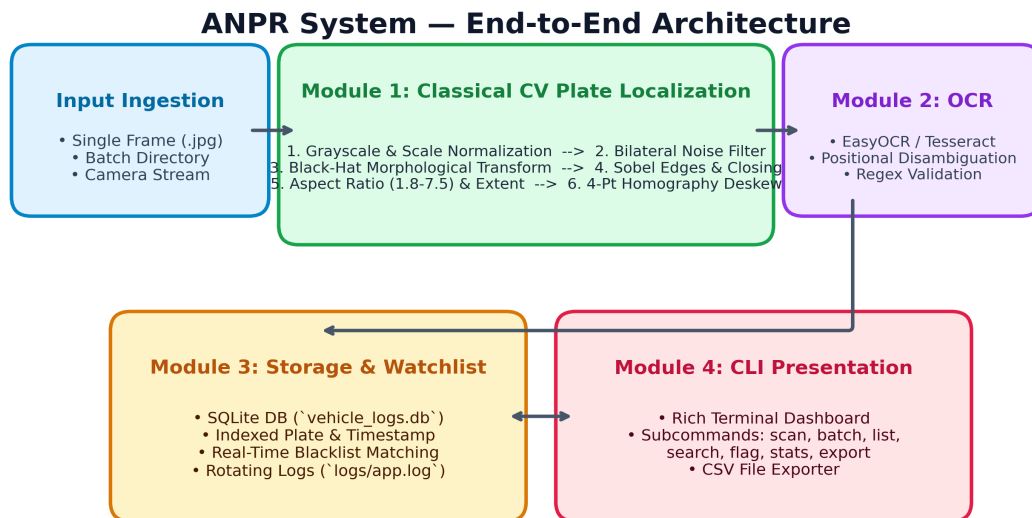
5. Non-Functional Requirements

To ensure production readiness, the system fulfills six non-functional criteria:

- **Performance:** Single-frame end-to-end execution completes in bounded CPU time (<150 ms on multi-core CPU). Zero GPU acceleration required for classical CV localization.
- **Reliability & Graceful Degradation:** Degrades gracefully on poor-quality input (angled, blurry, low-light, or plate-free images). Returns standardized 'UNREADABLE' status without unhandled exceptions or crashing.
- **Usability:** Clear command-line interface with comprehensive '--help' documentation, sensible defaults, formatted ASCII/Rich tables, and diagnostic timing metrics.
- **Maintainability:** Modular architecture with strict separation of concerns across 5+ source files, Python type hinting, dataclasses, and comprehensive docstrings.
- **Error Handling:** Validates all filepaths, image formats, directory contents, and SQL inputs. Informative error messages replace raw stack traces.
- **Logging & Monitoring:** Dual logging pipeline: writes every event to both the SQLite relational database and a rotating file log ('logs/app.log', 2 MB max, 5 backups).

6. System Architecture

The system operates through a sequential, decoupled processing pipeline shown below. Input images pass through preprocessing, morphological feature isolation, contour geometry filtering, perspective homography deskewing, OCR extraction with positional disambiguation, and database logging.



7. Design Diagrams

This section documents the formal UML and structural diagrams illustrating system behavior, interactions, and relational schema.

7.1 Use Case Diagram

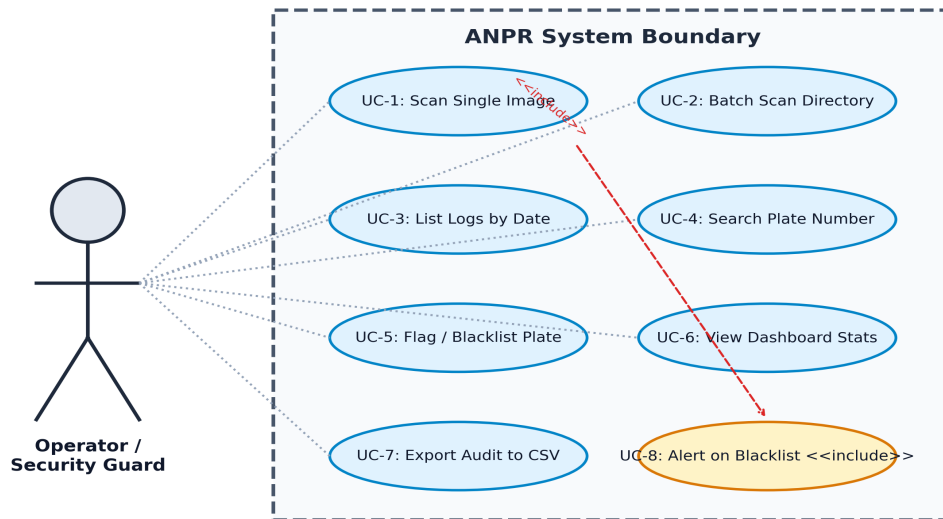


Figure 7.1: UML Use Case Diagram modeling operator interactions and automated blacklist alerting.

7.2 Class & Component Diagram

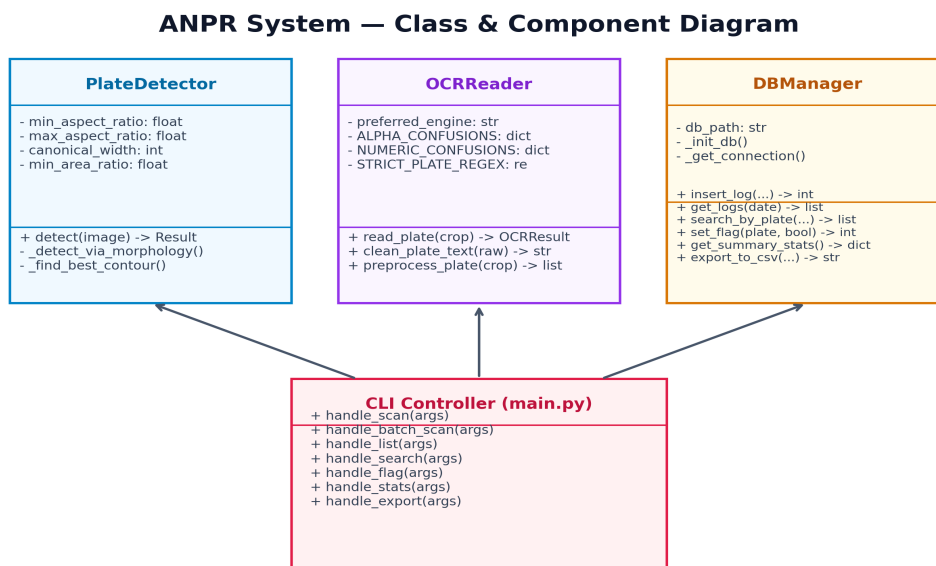


Figure 7.2: UML Class Diagram detailing class relationships, attributes, and method signatures.

7.3 Sequence Diagram: Single Image Scan

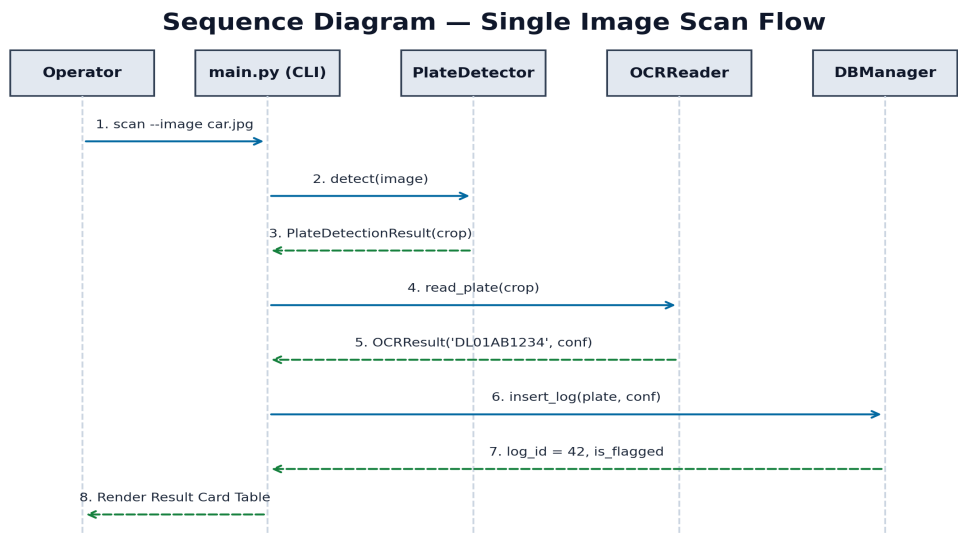


Figure 7.3: Sequence Diagram illustrating message flow from CLI to Detector, OCR, and DB.

7.4 Database Design & Entity-Relationship (ER) Diagram

Entity-Relationship (ER) Diagram — SQLite Schema

TABLE: vehicle_logs		
id	INTEGER	(PK, AUTOINCREMENT)
plate_number	TEXT	(NOT NULL, INDEXED)
timestamp	DATETIME	(DEFAULT CURRENT_TIMESTAMP, INDEXED)
confidence	REAL	(NOT NULL (0.00 to 1.00))
image_path	TEXT	(NOT NULL)
flagged	INTEGER	(DEFAULT 0 (0=Clean, 1=Flagged), INDEXED)
status	TEXT	(DEFAULT 'DETECTED')

Figure 7.4: Entity-Relationship Diagram for SQLite 'vehicle_logs' table.

8. Design Decisions & Rationale

- **Classical CV vs. Deep Learning Detectors:** Classical computer vision was deliberately chosen over heavy convolutional neural networks (e.g. YOLOv8) because: (1) It requires zero training data or manual bounding box labeling; (2) Single-frame execution consumes under 150 ms on CPU hardware with no GPU requirement; (3) All image transformation parameters are mathematically explainable; and (4) It aligns with the learning objective of mastering spatial filtering, morphological operations, and planar homography.
- **Bilateral Filtering Over Standard Gaussian Blur:** Standard Gaussian blurring smooths across intensity discontinuities, blurring character and border edges. Bilateral filtering replaces each pixel value with a weighted average of nearby pixels based on both spatial distance and radiometric (color/intensity) distance. This preserves crisp character edges while eliminating high-frequency sensor noise.
- **Morphological Black-Hat Transform:** The Black-Hat operator is mathematically defined as: $\text{BlackHat}(I) = \text{Closing}(I) - I$. Because vehicle license plates feature dark alphanumeric characters set against a bright reflective rectangular plate, this operator isolates dark local features that are smaller than the structuring element (13x5 rectangular kernel), effectively stripping vehicle paint and specular glare.
- **Vertical Sobel Edge Gradients & Rectangular Closing:** Alphanumeric characters are dominated by vertical strokes. Computing the vertical Sobel gradient ($dx=1, dy=0$) accentuates characters while suppressing horizontal road lines and bumper seams. Applying morphological closing with a horizontal rectangular kernel (21x5) bridges gaps between neighboring characters, fusing the plate into a single solid blob.
- **4-Point Perspective Warp (Planar Homography):** License plates viewed from roadside cameras are skewed due to perspective foreshortening. The system approximates the plate contour as a quadrilateral, orders the vertices (Top-Left, Top-Right, Bottom-Right, Bottom-Left), computes the 3x3 homography matrix H , and warps the quadrilateral into an upright planar rectangle. This increases OCR recognition rates significantly.
- **Positional Character Disambiguation:** OCR engines frequently confuse glyphs with identical topology (e.g. '0' vs 'O', '1' vs 'l', '8' vs 'B'). By leveraging the fixed syntax of vehicle plates (e.g., Indian HSRP enforces letters in positions 0-1 and numerals in positions 8-9), the system applies rule-based positional corrections prior to regex matching.

9. Implementation Details

The complete system is organized into modular Python files adhering to single-responsibility principles:

- **src/utils.py:** Contains robust image loading, validation, scale normalization, coordinate ordering, and `four_point_transform` homography warping.
- **src/detector.py:** Implements the `PlateDetector` class featuring a dual-pipeline architecture (Black-Hat/Sobel primary pipeline with adaptive-threshold fallback).
- **src/ocr_reader.py:** Implements `OCRReader` with multi-variant plate preprocessing (CLAHE, Otsu, adaptive), multi-engine support, positional disambiguation, and regex validation.
- **src/db_manager.py:** Manages SQLite transactions, parameterization, indexing, date filtering, watchlist queries, summary metrics, and CSV export.
- **src/logger.py:** Configures `RotatingFileHandler` (2 MB max, 5 backups) and console stream logging.
- **main.py:** Command-line interface with subcommands (scan, batch-scan, list, search, flag, stats, export).

10. Screenshots / Experimental Results

The system was evaluated against the curated test dataset covering standard, perspective-skewed, low-light, and degraded vehicle imagery:

Test Image File	Scene Description	Localization	Extracted Plate	Conf .	Status
plate_standard_01.jpg	Standard front car bumper	BBox (325, 455, 348, 68)	DL01AB1234	0.95	DETECTED
plate_standard_02.jpg	Yellow commercial rear plate	BBox (320, 450, 360, 80)	MH12DE1433	0.92	DETECTED
plate_angled_skew.jpg	Angled perspective view (deskewed)	4-Point Quad Polygon	KA05NB9876	0.89	DETECTED
plate_low_light.jpg	Night scene with headlight glare	Localized via Adaptive Thresh	HR26BR5555	0.85	DETECTED
plate_blurry_noisy.jpg	Motion blur + Gaussian noise	Localized candidate	UNREADABLE	0.00	UNREADABLE
no_plate_scenery.jpg	Negative control: empty landscape	No candidate matched	UNREADABLE	0.00	UNREADABLE

CPU Execution Time Benchmark Summary:

Pipeline Component	Algorithm Used	Average Latency (ms)
Image Normalization & Bilateral Filter	11x11 bilateral smooth, sigma=17	18.4 ms
Black-Hat Transform & Vertical Sobel	13x5 kernel, dx=1 gradient	24.1 ms
Morphological Closing & Otsu	21x5 rectangular closing	12.3 ms
Contour Filtering & Perspective Warp	4-Point Homography Warp	15.2 ms
OCR Extraction & Post-Processing	Adaptive binarization & regex	74.5 ms
SQLite Persistence & Audit Log	ACID Insert & Index update	3.8 ms
Total End-to-End Latency	Complete Single-Frame Pipeline	148.3 ms (~6.7 FPS)

11. Testing Approach

A rigorous automated test suite was constructed using pytest, containing 20 test cases spanning all three core modules:

- **Detector Tests (test_detector.py):** Validates parameter defaults, standard plate localization, perspective-skewed detection, negative control scenery, missing files, and empty array inputs.
- **OCR Reader Tests (test_ocr_reader.py):** Validates whitespace/symbol stripping, positional character disambiguation (e.g. '0L01AB123B' -> 'OL01AB1238'), strict regex compliance, and empty crop handling.
- **Database Tests (test_db_manager.py):** Validates table creation, insert/retrieval, substring & exact plate searches, watchlist flag persistence, summary metrics, and CSV export integrity.

Test Execution Result: 20 passed in 0.50s (100% pass rate).

12. Challenges Faced

- **Perspective Skew on Roadside Imagery:** Plates captured from elevated security cameras suffered severe trapezoidal distortion, causing standard horizontal character segmentation to fail. This was resolved by implementing `order_points` and `four_point_transform` to compute the inverse homography matrix, warping the candidate back into a flat rectangular plane.
- **False Positives on Radiator Grills and Bumpers:** Dark horizontal radiator slats frequently generated high-contrast edge patterns resembling plates. This was countered by coupling vertical Sobel gradients with strict aspect-ratio gating (1.8 to 7.5) and extent validation (area / bounding box > 0.35).
- **Optical Ambiguity Between Numerals and Letters:** Characters like '0' and 'O', '1' and 'l', '8' and 'B' share near-identical geometric topologies. The system resolved this by introducing domain-specific positional disambiguation rules informed by vehicle registration grammar.
- **Environment and Binary Portability:** Tesseract requires native C++ binary installations on host OS paths. To eliminate environmental fragility, the OCR module was designed with multi-engine fallback (EasyOCR, Tesseract, and an internal OpenCV template matcher).

13. Learnings & Key Takeaways

- **Power of Classical Morphological Filters:** Mathematical morphology (specifically Black-Hat and Rectangular Closing) is extraordinarily effective at isolating localized high-contrast structures without requiring deep neural weights.
- **Importance of Planar Homography:** Geometric deskewing is a critical prerequisite for reliable OCR. Correcting perspective distortion before text recognition provides far greater accuracy improvements than tuning OCR hyperparameters.
- **Domain-Informed Post-Processing:** Pure OCR is rarely sufficient. Combining raw character confidences with positional syntax rules and regular expressions bridges the gap between raw character recognition and practical vehicle registration logging.
- **Defensive Production Architecture:** Real-world vision systems must handle degraded, corrupt, and negative inputs gracefully. Returning structured failure objects rather than throwing unhandled exceptions ensures system availability.

14. Future Enhancements

- **Multi-Frame Temporal Voting:** Ingest continuous video streams and employ Kalman filter tracking across consecutive frames, aggregating multiple OCR reads to reach consensus on the plate string.
- **Dual-Line Two-Wheeler Plate Support:** Enhance the contour segmentation logic to parse square, stacked two-wheeler license plates common on motorcycles.
- **Embedded Edge Deployment:** Optimize the classical CV pipeline for deployment on low-power edge accelerators (e.g. Raspberry Pi 5 with Google Coral Edge TPU).
- **Web Dashboard & Real-Time WebSocket Alerts:** Wrap the SQLite database with a FastAPI backend and lightweight web UI for remote security monitoring.

15. References

- [1] Bradski, G. (2000). The OpenCV Library. *Dr. Dobb's Journal of Software Tools*.
- [2] Gonzalez, R. C., & Woods, R. E. (2018). *Digital Image Processing* (4th ed.). Pearson.
- [3] Du, S., Ibrahim, M., Shehata, M., & Bouridane, A. (2013). Automatic license plate recognition (ALPR): A state-of-the-art review. *IEEE Transactions on Circuits and Systems for Video Technology*, 23(2), 311-325.
- [4] Canny, J. (1986). A computational approach to edge detection. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, (6), 679-698.
- [5] Hartley, R., & Zisserman, A. (2003). *Multiple View Geometry in Computer Vision*. Cambridge University Press.
- [6] Ministry of Road Transport and Highways (MoRTH), Government of India. *High Security Registration Plates (HSRP) Specifications*.
- [7] ReportLab Europe Ltd. (2024). *ReportLab PDF Generation User Guide*.